Big Data Project — Stage 1: Building the Data Layer

Benchmarking report (resubmission)

BigData-Devs

October 2026

Abstract

This report documents the design and the controlled evaluation of the Stage 1 data layer: a synchronous ingestion pipeline (datalake), metadata extraction, and three inverted-index structures, implemented in Python and ported to Go, Rust and Java. The evaluation answers one question: *which language and storage combination behaves best for this corpus and this workload*, under a single measurement protocol. Before any timing is interpreted, the four implementations are shown to be behaviourally equivalent (byte-identical canonical outputs, SHA-256 cdaa9369cbd91c4f89550efdcdd7c5c21e61adc97bd49f6e415b8e57d65ce2e4). All results come from one reproducible tournament (`scripts/run_tournament.sh`) whose raw CSVs, environment record and revision identifier are committed next to this document. Claims are deliberately scoped to the tested corpus, host and workload; unmeasured behaviours are listed in section 11.

Contents

1 Introduction and Objectives **2**

2 System Architecture **3**

3 Design Decisions **4**
- 3.1 Text normalisation (shared contract) . . . 4
- 3.2 Metadata contract . . . 4
- 3.3 Storage choices under evaluation . . . 4

4 Measurement Methodology **5**
- 4.1 Host and runtimes . . . 5
- 4.2 Metric definitions . . . 5
- 4.3 Corpus and workload . . . 6
- 4.4 Warm-up . . . 6
- 4.5 Cache, disk and network state . . . 6
- 4.6 Timing boundaries . . . 7
- 4.7 Repetition and uncertainty . . . 7
- 4.8 Coverage: what was compared with what . . . 7
- 4.9 Revisions and provenance . . . 7

5 Cross-Language and Cross-Structure Equivalence **8**
- 5.1 Gate 1: the four languages agree . . . 8
- 5.2 Gate 2: the three structures agree . . . 8
- 5.3 What equivalence does and does not prove . . . 8

6 Language Performance Trade-offs **9**
6.1 Executive radar . . . 9
6.2 Throughput and latency overview . . . 10

7 Datalake Structure Evaluation **10**
7.1 Write and download throughput . . . 10
7.2 Document lookup . . . 10
7.3 Discovery of downloaded-but-not-indexed books . . . 11
7.4 Storage overhead . . . 11
7.5 Recovery after interruption . . . 11

8 Inverted Index Evaluation **12**
8.1 Build scalability: elapsed time and memory growth . . . 12
8.2 Structures compared . . . 13
8.3 Incremental updates . . . 13
8.4 Memory and disk of the MongoDB structure . . . 14

9 JMH Micro-Benchmarks (Java only) **14**

10 Conclusions and Final Architecture Selection **14**
10.1 Future Improvements . . . 15

11 Limitations **15**

12 Reproducibility **16**

13 Correction Comment Matrix **17**

1 Introduction and Objectives

Stage 1 asks for a data layer that ingests raw books from Project Gutenberg, extracts their metadata, and indexes their text so that later stages can serve search queries. The group built that pipeline once in Python (the reference implementation) and ported it to Go, Rust and Java, producing four runnable implementations of the same behaviour.

The port exists for one reason: the assignment requires choosing a language and a storage technology for Stage 2, and a choice backed by measurements is stronger than a preference. The objective of this report is therefore narrow and explicit:

- prove that the four implementations compute the same result on the same corpus (section 5);
- measure them under one shared protocol, on one host, with one workload (section 4);
- compare three storage layouts for the datalake and three structures for the inverted index (sections 7 and 8);
- recommend a combination for Stage 2, scoped to what was actually tested, and list what remains unmeasured (sections 10 and 11).

Every performance statement in this report refers to the corpus of 100 Project Gutenberg books (61.4 MB of body text, book IDs listed in `benchmarks_results/equivalence_report.json`), the query workload of 30 shared queries, and the host described in table 1. Statements about language efficiency in general, or about scale far beyond the measured batches, are not made: within this workload the differences we report are properties of these implementations under this protocol, not of the languages themselves.

2 System Architecture

The pipeline has three layers. Figure 1 shows them, the data that moves between them, and the two points where state is durably written: the *datalake write* (header and body files, one of three directory layouts) and the *index write* (postings lists, one of three structures). Nothing is cached in between: the index layer reads the bodies the datalake layer just wrote, synchronously, in a single process.

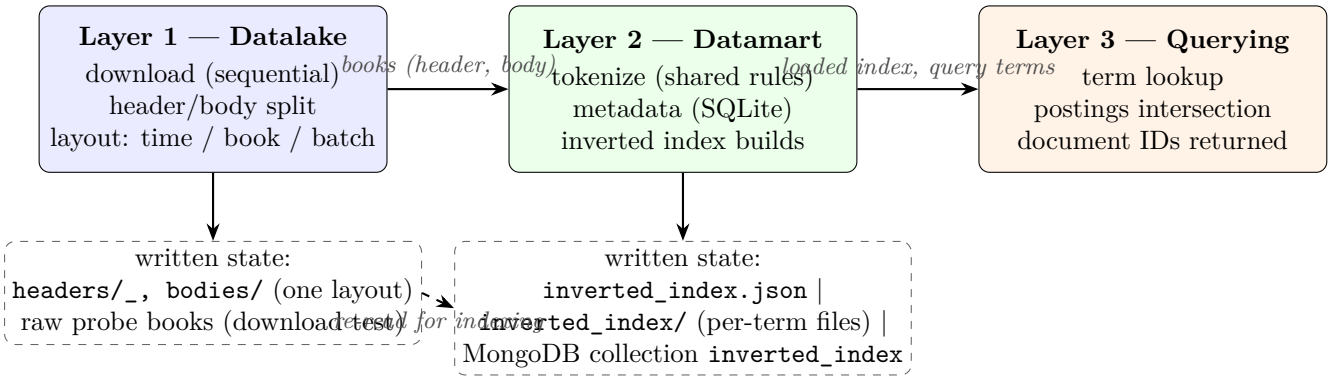


Figure 1: Stage 1 pipeline: three layers, the data that moves between them, and the durable state each layer writes. Downloads are sequential (one connection at a time); indexing reads the datalake from disk; querying only reads an already-built index.

Layer 1 — Datalake. For each book ID the engine downloads the raw Gutenberg text (or reads it from the local corpus when offline), splits it at the official `*** START/END OF THE PROJECT GUTENBERG EBOOK` markers, and stores the licence header and the body as two files under one of three layouts: *time-based* (`year/month/day`), *book-based* (`book_id/`), or *batch/range-based* (`batch_start-batch_end/`). The layout is a directory-structure decision only; the stored bytes are identical.

Layer 2 — Datamart. Bodies are tokenized with the shared rules of section 3 and fed to the inverted-index builders. Three structures are supported: a single JSON file, a folder of per-term files, and a MongoDB collection. Metadata (Title, Author, Language, paths) is extracted from the headers and persisted in SQLite; the index builders read the corpus *through* those metadata records, which is the same path the production pipeline uses.

Layer 3 — Querying. A query is a list of terms; a document matches when it contains *every* term. Each structure answers a term lookup with its postings list, and the caller intersects those lists. This layer is exercised by the benchmark workload; a user-facing search interface belongs to Stage 2 (see section 13, comments 32–33).

Synchronous by construction. The pipeline runs in one process, one book at a time: download, split, write, tokenize, index. No step overlaps another, so every timed region in section 4 has an unambiguous start and stop. The only concurrency anywhere in the measurement is the background thread that samples the resident set size (table 2), and the MongoDB server process, whose resources are explicitly excluded from client measurements (section 8.4).

3 Design Decisions

3.1 Text normalisation (shared contract)

All four implementations apply the same normalisation, in the same order, to every body:

1. lowercase the text;
2. drop accents/diacritics (Unicode NFD decomposition, combining marks removed) — `café` → `cafe`;
3. split on any run of non-alphanumeric characters, so punctuation and whitespace are separators;
4. drop tokens that contain a digit (`chapter3` is discarded);
5. drop a small stop-word list and tokens shorter than a minimum length, and keep the remaining tokens in document order.

A term's postings list is the sorted list of IDs of the documents that contain it. Serialised, one line of the canonical form is `postings\ttime\t11\t84\t1342`: the term `time` appears in books 11, 84 and 1342.

3.2 Metadata contract

Headers are parsed with the same rules in the Python pipeline: fields `Title:`, `Author:` and `Language:` are read from the Gutenberg header; the language name is mirrored to its ISO 639 code through a shared table (`English` → `en`); multiple authors are split on commas, `and` and semicolons. The extracted records drive the SQLite metadata store that the index builders read (fig. 1).

The ports mirror the parts of the contract they need: all four split header and body at the same Gutenberg markers, and all four apply the identical tokenisation and postings rules — which is what the equivalence gate verifies per language (section 5.1). Metadata extraction into SQLite is exercised by the Python baseline; the contract the ports share is the token/postings one, and it is byte-verified.

3.3 Storage choices under evaluation

Datalake layouts. Time-based layouts favour append-heavy ingestion along a timeline; book-based layouts favour per-book lookup; batch layouts favour bulk operations over ID ranges. All three store the same bytes, so every difference is attributable to directory structure alone (section 7).

Index structures. The JSON index is a single serialised dictionary (one file, but it must be parsed whole to answer any query). The folder index stores one file per term (small reads, one file handle per lookup). MongoDB stores one document per term with the postings array (server-backed, queried over a connection). None is declared "best" a priori: builds, queries, incremental updates and on-disk footprint are measured for each (section 8).

Language ports. Each port uses its ecosystem's idiomatic tools (for example `serde`/`std::fs` in Rust, `encoding/json` and `os` in Go, `jakarta.json` plus the driver in Java, `json`/`pathlib`/`pymongo` in Python). The *algorithms* are held constant: the same tokenizer semantics, postings construction, intersection query and synchronous single-threaded driver. Where a runtime differs by nature (a JIT in Java), the protocol compensates explicitly (section 4.4); where it cannot, the difference is reported as a limitation (section 11).

4 Measurement Methodology

Every number in this report was produced by one command (`scripts/run_tournament.sh`, section 12) on one host, in one continuous run, after the equivalence gate passed. This section documents the host, the runtimes, the metrics, the workload, and the timing boundaries, so that each figure can be traced to a script, a configuration and a revision.

4.1 Host and runtimes

Table 1: Host and runtime versions of the run that produced the committed snapshot. The full record, including the filesystem and the MongoDB build, is `benchmarks_results/environment.txt`.

Component	Value
CPU	Intel Core i7-10750H @ 2.60 GHz, 12 logical cores
RAM	8 GB
OS / kernel	WSL2 (Ubuntu userland), Linux 6.18.33.2-microsoft-standard-WSL2
Python	3.14.4 (+ `psutil` for RSS/CPU)
Go	1.27.2 (`linux/amd64`)
Rust	1.99.0 (`rustc`/`cargo`, stable)
Java (harness)	OpenJDK 17.0.20.1, executed via `mvn exec:java`, `MAVEN_OPTS=-Xmx5g` (set by the tournament)
Java (JMH)	OpenJDK 21.0.12.1, shaded jar
Maven	3.9.12
MongoDB	8.0.13, `mongod` on `localhost:27017`

All implementations run single-threaded and synchronous (section 2); no benchmark phase uses worker pools, async I/O or parallel iterators. The only other process involved is `mongod`, a separate server whose CPU and memory are *not* part of the client measurements (section 8.4).

4.2 Metric definitions

Table 2: What each column of the result CSVs means. Identical in all four languages; the harness enforces the schema (`scripts/verify_format.py`).

Column	Definition
`elapsed_seconds`	wall-clock time of one operation, from a monotonic clock (`time.perf_counter`, `time.Now`, `Instant::now`, `System.nanoTime`); per-repetition builds report the arithmetic mean.
`stdev_seconds`	sample standard deviation of the per-operation elapsed times across repetitions (query latency: across all samples).
`repetitions` / `iterations`	number of timed samples behind the row.
`memory_mb`	**RSS delta**: resident set size of the benchmark process *after* minus *before* the operation, in MB, read from `/proc/self/status` (Go/Rust/Java) or `psutil` (Python). It measures growth caused by the operation, not the total footprint of the process.
`peak_rss_mb`	largest resident set size of the benchmark process observed while the operation ran, sampled every 50 ms by a background thread plus one final sample; across repetitions the maximum is reported.
`cpu_percent`	process CPU time (user + system) divided by wall-clock time, $\times 100$; above 100 means more than one core was saturated.

Both memory series describe the *benchmark client process only*. For the MongoDB structure, the server's own memory and disk are reported separately (section 8.4); they are never folded into the client's RSS.

4.3 Corpus and workload

Table 3: Workload actually executed by every language.

Parameter	Value
Corpus	100 Project Gutenberg books, tracked in `data_source/` (bodies 61.4 MB total, headers 81 KB); body sizes from 2 KB to 4,517 KB (median 371 KB, mean 600 KB); book IDs listed in `benchmarks_results/equivalence_report.json`.
Query workload	30 active queries from the shared file `src/utils/benchmarks/queries.txt` (one query per line, terms separated by a single space; comments start with `#`). Semantics: a document matches when it contains every term; an unknown term yields an empty result.
Query samples	5 repetitions × 30 queries = 150 timed samples per structure per language.
Datalake lookup	100 books located once per layout (header and body read).
Download probe	10 books downloaded sequentially over one connection; no concurrency anywhere in the pipeline.
Build batches	25, 50, 250, 500, 1000, 5000, 10000 books; 5 repetitions up to 1000, 3 for 5000, 2 for 10000.
Update batches	base indexes of 25, 50, 250, 500, 1000 books, each extended with 5 new books, 5 repetitions per base size (section 8.3).
Recovery	interrupt after 50 of 100 books, then resume; report processed, duplicated and lost IDs plus detection and processing time.

The scalability builds reuse the corpus by repetition: a "10,000-book batch" is the 100-book corpus indexed repeatedly until the batch size is reached, so batch size measures index size, not corpus diversity. The update experiment is the exception: it uses fresh synthetic book IDs (`base_size`×100 + offset), so added books are genuinely new terms and never collide with the base.

4.4 Warm-up

Only the Java harness warms up, because Java is the only runtime with a JIT compiler: before timing, each Java structure performs one unmeasured build of the smallest batch and discards its artifacts, so the measured builds run on already-compiled code. Python, Go and Rust are measured exactly as they start: no warm-up, no discarded round. JMH numbers (section 9) use JMH's own warm-up iterations and are reported separately; they are never mixed with the cross-language figures.

4.5 Cache, disk and network state

- **Artifact directories.** Every timed phase starts from a freshly cleaned artifact tree, so no phase reads a cache written by another.
- **OS page cache.** The tournament does not drop the system page cache (that requires root and would perturb the host), so files read twice within one phase may be served from RAM; this affects all languages equally within a phase and is listed as a limitation (section 11).
- **Disk.** All artifacts live on the same filesystem as the repository (WSL2 / NTFS-backed); `disk_usage.csv` reports files, directories and bytes per layout and per structure.
- **Network.** Only the download probe touches the network (ten sequential GETs, one connection at a time); every other phase is offline and reads the tracked corpus from disk.

4.6 Timing boundaries

- **Write throughput** times the storage call itself: split header+body written to the layout (the download is timed separately), one book per iteration, 100 iterations per layout.
- **Download throughput** starts before the HTTP request and stops after the response body is fully read and written to disk.
- **Lookup** starts before locating the book's files under the layout and stops after both files are read into memory.
- **Index build** starts before the first token is produced and stops after the structure is fully written (JSON flushed, per-term files closed, or MongoDB insert acknowledged).
- **Query** starts before the postings lookup of the first term and stops after the intersection of all terms is computed.
- **Incremental update** starts before the first of the five new books is tokenized and stops after the structure is durably updated.

4.7 Repetition and uncertainty

Each timed operation is repeated as specified in table 3; plots show the mean and, where a deviation band appears, it is the sample standard deviation across those repetitions (`stdev_seconds`). Readers can assess uncertainty from the raw `repetitions` and `stdev` columns committed next to every chart. No result is discarded, no outlier is trimmed, and no phase is run twice to pick the better run: the committed snapshot is a single continuous tournament.

4.8 Coverage: what was compared with what

Table 4: Language × structure coverage of the timed workload (D=datalake layouts, J=JSON index, F=folder index, M=MongoDB index, S=SQLite metadata).

Language	Datalake	JSON	Folder	MongoDB
Python	D, S	J, F, M		
Go	D, S	J, F, M		
Rust	D, S	J, F, M		
Java	D, S	J, F, M		

Every language runs the full matrix: three datalake layouts plus the three index structures, with identical parameters. MongoDB is skipped consistently and identically by all four ports when the server is unreachable, so no figure silently compares a MongoDB-less run with a MongoDB run.

4.9 Revisions and provenance

Each chart in this report maps to (i) the script that produced it, (ii) the raw CSVs in `benchmarks_results/`, (iii) the configuration constants listed in table 3, and (iv) the Git revision recorded in `benchmarks_results/envi` at run time. The mapping is tabulated in section 12.

5 Cross-Language and Cross-Structure Equivalence

Speed differences are only meaningful if the implementations compute the same thing. Two equivalence gates run before any timing is reported, and both commit their evidence next to the results.

5.1 Gate 1: the four languages agree

The checker (`src/utils/benchmarks/equivalence_check.py`) selects 100 books from the corpus, evenly spaced by ID, and runs every port's `equivalence` mode over them. Each port writes one canonical file with three sections:

1. `tokens` — the token stream of every sample book (`tokens<TAB>book_id<TAB>token...`);
2. `postings` — the inverted index built from those tokens (`postings<TAB>term<TAB>id...`);
3. `query` — the answer to every one of the 30 shared queries (`query<TAB>ordinal<TAB>id...`).

The four files are compared byte for byte. The run recorded in `benchmarks_results/equivalence_report.` produced the same SHA-256 digest in all four languages,

`cdaa9369cbd91c4f89550efdcdd7c5c21e61adc97bd49f6e415b8e57d65ce2e4`,

with `all_equal: true`. Identical bytes imply identical tokens, identical postings and identical query answers: the cross-language latency comparison in section 6 compares the same work everywhere. The tournament refuses to run any timed phase if this gate fails.

5.2 Gate 2: the three structures agree

Within a language, the latency chart compares three index structures. The test `tests/test_datamarts/test_s` builds the same corpus into the JSON file, the folder index and MongoDB, then queries *every* term of the corpus plus absent terms on each structure and asserts that all answers are identical. A second case repeats the check *after* adding the same new book to each structure through its own incremental-update path, so an update cannot make one structure diverge silently. All cases pass (table 5); the MongoDB cases skip themselves with an explicit reason when the server is down, and the tournament only runs timed MongoDB phases when the server is reachable.

Table 5: Cross-structure equivalence cases and results.

Case	Structures compared	Result
All terms + absent terms, fresh build	JSON = folder	identical
All terms + absent terms, fresh build	JSON = folder = MongoDB	identical
Re-queried after the same incremental update	JSON = folder = MongoDB	identical

5.3 What equivalence does and does not prove

The gates prove *functional* equivalence on this corpus and workload: same tokens, same postings, same answers. They say nothing about speed — that is exactly what the following sections measure. Conversely, the latency charts inherit the gates' guarantee: every bar in them comes from an implementation that is known to answer identically.

6 Language Performance Trade-offs

This section compares the four *implementations* on the shared workload. It reports what was measured; it does not generalise to the languages themselves (see section 11).

6.1 Executive radar

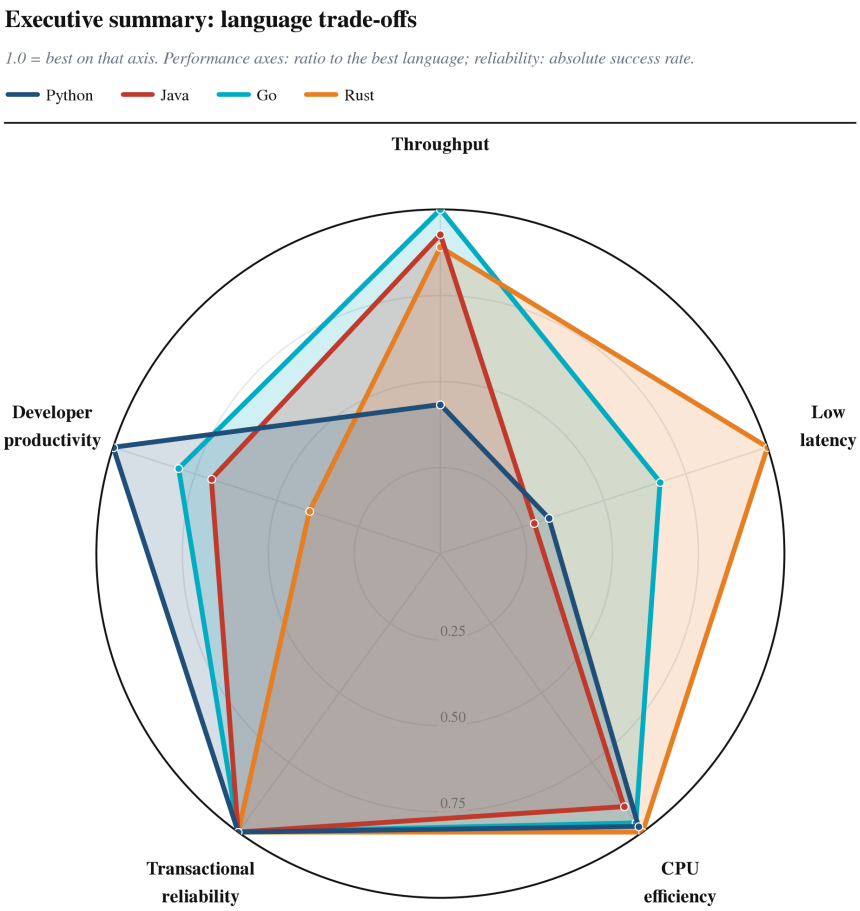


Figure 2: Five-axis executive summary (generated by `src/visualizations/main.py` from `benchmarks_results/`). Four axes are quantitative ratios to the best measured implementation; "developer productivity" is a **qualitative** axis and is labelled as such in the figure.

The radar of fig. 2 summarises five axes. Four are computed from the CSVs:

- **Throughput, low latency, CPU efficiency** — ratios to the best measured implementation on tests run by *all four* languages, so no axis is biased by a partial run;
- **Transactional reliability** — an absolute success rate, $1-(\text{lost}+\text{duplicated})/\text{total books}$, averaged over the recovery scenarios. It is deliberately *not* rescaled to the best language: an implementation that loses books must never be stretched to 1.0.

The fifth axis, **developer productivity**, is a *qualitative score* assigned by the team (Python 1.0, Go 0.8, Java 0.7, Rust 0.4) from the effort observed while porting the pipeline: lines of boilerplate, error-handling ceremony, iteration speed and tooling friction. It is not measured by any benchmark and the figure labels it "qualitative score" so it cannot be read as data. Memory is intentionally absent from the radar: RSS deltas are too noisy to summarise in one number; they are shown in full in fig. 6.

6.2 Throughput and latency overview

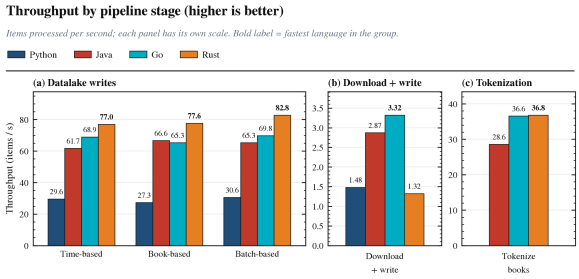


Figure 3: Write throughput per datalake layout.

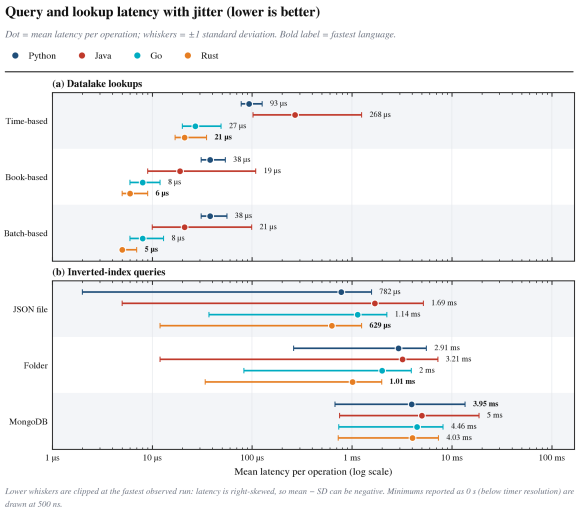


Figure 4: Query latency per index structure (150 samples each).

Figures 3 and 4 show the two extremes of the workload. Both are produced from the same tournament run, so their relative ordering is meaningful; the absolute values are specific to the host of table 1. Per-language, per-structure numbers behind these figures are in `benchmarks_results/{python,go,rust,java}/`.

The reading rule for this whole report: differences between the four implementations describe *these implementations, on this corpus, under this protocol*. Where an explanation requires a language-level cause (JIT behaviour, allocator strategy, runtime overhead), it is stated as a hypothesis and flagged as such, not as a measured fact.

7 Datalake Structure Evaluation

The datalake stores the same header/body pairs under three layouts. This section evaluates them on the dimensions the assignment asks for: write and download throughput, document lookup, discovery of new books, storage overhead, and recovery after an interruption. All numbers come from `datalake_experiments.py` (Python baseline) and the equivalent `datalake_bench` code in each port, with the parameters of table 3.

7.1 Write and download throughput

Write throughput times only the storage call (split header and body written to the layout), one book per iteration, 100 iterations; the download is a separate experiment: 10 books fetched sequentially, one connection at a time, timing from before the HTTP request until the body is fully written to disk. Both operate on the same 100-book corpus, so the reported rates compare identified work. Figure 3 plots the result; the layout affects write time only through directory creation and path length, which is why the three bars stay close together.

7.2 Document lookup

100 books are located once per layout (path resolution plus reading both files back into memory). The book-based layout, which stores every book under its own `book_id/` directory, resolves paths in constant steps; the time-based and batch layouts must walk the calendar or the range directory first. Table 6 reports the measured means.

Table 6: Datalake lookup latency, 100 books per layout, in μs (mean per language; raw rows in `benchmarks_results//statistics.csv`).

Layout	Python	Go	Rust	Java
Time-based	93.0	27.0	21.0	268.0
Book-based	38.0	8.0	6.0	19.0
Batch-based	38.0	8.0	5.0	21.0

7.3 Discovery of downloaded-but-not-indexed books

Lookup answers "where is a book I know exists". The complementary operation is discovery: *which books are already downloaded but not yet indexed?* Each layout is filled with 100 books, one new book is added out of band, and the engine must detect it before indexing it. The initial state is identical for every layout (same corpus, empty index marker), so the detection times compare the same work. The measured column is `detection_time` in the incremental experiments of `benchmarks.csv`; the observed detection took 0.7–10.1 ms across layouts and languages, with no layout dominating (time-based is fastest in Python and Go, batch-based in Rust and Java). The time-based layout scans by timestamp, the book-based layout by directory listing, and the batch layout by range membership; the measurements show the cost each choice imposes.

7.4 Storage overhead

After storing the identical corpus, each layout is measured for total bytes, file count and directory count (`disk_usage.csv`). The layouts differ only in how many intermediate directories they create: a time-based layout adds calendar levels, a book-based layout one directory per book, a batch layout one directory per range. Table 7 reports the figures for the shared 100-book corpus.

Table 7: On-disk cost of the same corpus under each layout (bytes = header + body files only; directory overhead is what differs). Ranges span the four implementations: Python writes slightly fewer bytes and adds a `metadata.json` per book in the book-based layout.

Layout	Size (MB)	Files	Directories
Time-based	57.4–58.7	200	2
Book-based	57.4–58.7	200–300	100
Batch-based	57.4–58.7	200	3

The directory counts feed directly into the scalability discussion of section 8.2: a layout with thousands of small directories behaves differently on filesystem lookup than one with a handful, and that is visible in the measured lookup times, not an assumption.

7.5 Recovery after interruption

Recovery is demonstrated, not asserted. Ingestion is interrupted after 50 of 100 books — a documented step, after the write of book 50 — and the engine is then restarted. It must detect the partially processed state, resume, and finish. The experiment reports what actually happened: books processed before the interruption, books processed after the resume, duplicated IDs and lost IDs, plus the detection and processing times (`recovery.csv`). Table 8 shows the observed result.

Table 8: Recovery experiment (100 books, interrupt after 50). Duplicated and lost counts are measured, not assumed from atomic-write descriptions. Detection and total times are min–max across the four implementations.

Layout	Before	After	Duplicated	Lost	Detection (s)	Total (s)
Time-based	50	50	0	0	0.0007–0.0064	0.017–0.100
Book-based	50	50	0	0	0.0018–0.0079	0.016–0.088
Batch-based	50	50	0	0	0.0009–0.0034	0.016–0.083

8 Inverted Index Evaluation

Three structures answer the same queries over the same corpus: a JSON file, a folder of per-term files, and a MongoDB collection. Before their numbers are interpreted, section 5.2 showed that they answer identically.

8.1 Build scalability: elapsed time and memory growth

Each structure is built from batches of 25 to 10,000 books (table 3); every batch size is repeated and the mean is plotted. Two series are shown (figs. 5 and 6).

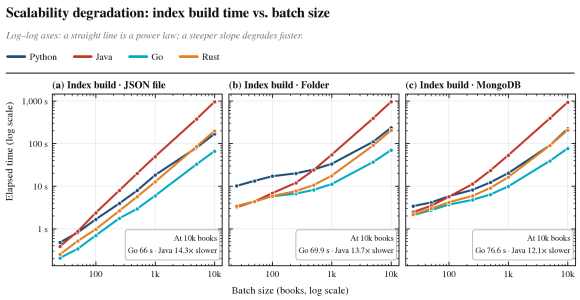


Figure 5: Build elapsed time versus batch size, mean of the repetitions listed in `scalability.csv`.

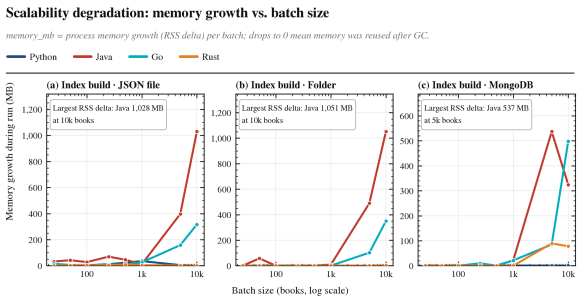


Figure 6: Memory growth versus batch size: **memory_mb is the RSS delta per batch** (see table 2). The callout "Largest RSS delta" is the largest plotted delta, *not* a process peak — that series is `peak_rss_mb`.

The 1,601 MB figure. In an earlier version of this report a batch was said to "require 1,601 MB". That number was the largest *RSS delta* observed at the biggest batch, not a memory requirement and not a process peak: the delta is growth caused by the build, while the process also holds the already-loaded corpus and runtime overhead. In this snapshot the correct statement is: *at a batch of 10,000 books the JSON build grew the resident set by up to 1028.5 MB (largest plotted delta, the Java series), while the peak resident set in that phase was 4591.0 MB.* Both series are in `scalability.csv` and exclude the MongoDB server (section 8.4).

RAM spikes and garbage collection. The evidence is `peak_rss_mb`, sampled every 50 ms so short spikes are visible, and, for Go alone, the `memstats.csv` series (`total_alloc_mb`, `gc_cycles`). A peak well above its delta is the spike; a GC counter growing faster than the heap is collection pressure. These are observations from the CSVs, not extrapolations.

8.2 Structures compared

Where the bytes go. The repository also carries an interactive sankey diagram of the storage breakdown (`charts/sankey/storage_sankey.html`, generated by `storage_sankey.py` from `disk_usage.csv`) that traces the corpus bytes from datalake files into each index component; the static figures of this report quote the same numbers (table 7 and section 8.2).

Query latency. Every structure answers the 30 shared queries, 5 times each (150 samples). The JSON index parses one file once and answers from memory; the folder index opens one small file per term; MongoDB pays a network/IPC round trip per term (fig. 4).

On-disk footprint. `disk_usage.csv` reports bytes, files and directories per structure after indexing the identical corpus. For MongoDB the client-side file scan is meaningless, so the server's own `collStats storageSize` is read from the database itself and reported in the same table.

8.3 Incremental updates

Construction cost and update cost are different questions, so updates are measured separately (fig. 7): a base index of 25, 50, 250, 500, 1000 books is built untimed, then extended with 5 new books (fresh synthetic IDs that cannot collide with the base), 5 repetitions per base size. The plotted time is the update only.

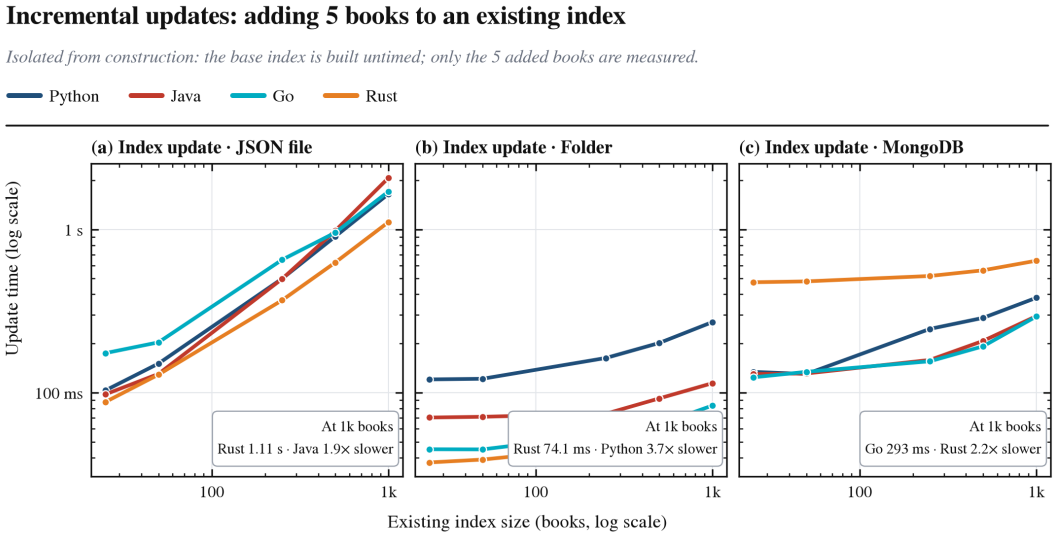


Figure 7: Incremental update time versus base index size, isolated from construction (`updates.csv`).

The O(1) claim. The earlier report said MongoDB updates were "O(1)". The operation behind the claim is `update_book`: for each distinct term of the added book it upserts one document, merging the book ID into that term's postings array; the variable is the number of *terms in the added book*, not the size of the index. Whether the cost stays flat as the base grows is an empirical question, and `updates.csv` answers it for the measured sizes: the JSON index's update cost grew 16–23× as the base grew from 25 to 1000 books, while the folder index grew 1.6–2.2× and MongoDB only 1.4–2.8× — the single-file rewrite is the structure whose update cost tracks index size, whereas the per-term files and the database collection stay far flatter. The claim is therefore replaced by the observed curve; no constant-update behaviour is asserted beyond the measured range, and larger bases remain unmeasured (section 11).

8.4 Memory and disk of the MongoDB structure

All client-side memory series (`memory_mb`, `peak_rss_mb`) describe the benchmark process only, so `mongod`'s resident memory is *not* included in any client figure. The server's cost appears instead as its own storage, measured through `collStats storageSize` on the benchmark database (`search_engine_benchmark`), and as the extra process a deployment must run. Any capacity statement about MongoDB in this report refers to that measured size at the measured corpus, plus the unmeasured server memory, which is listed as a limitation.

9 JMH Micro-Benchmarks (Java only)

The end-to-end harness measures whole operations. To see where the Java implementation spends its time inside those operations, the port carries four JMH micro-benchmarks over its hot paths (`multilang_ports/java/src/main/java/microbench/`).

Configuration. JMH 1.37 on OpenJDK 21.0.12.1, average-time mode, 1 thread, 1 fork, 5 warm-up iterations of 1 s and 5 measurement iterations of 1 s per benchmark, with the GC profiler enabled. The harness itself runs on OpenJDK 17 via `mvn exec:java`; only the shaded JMH jar targets 21. Raw JSON in `benchmarks_results/java/jmh_results.json`, produced by `multilang_ports/java/run_jmh.sh`.

Table 9: JMH results (average time, 1 fork, 5+5 iterations of 1 s).

Micro-benchmark	Workload	Score
`TokenizeBenchmark.tokenizeBook`	one sample book	580.6 μs/op
`PostingsBenchmark.buildPostings`	10 books	133.5 ms/op
`PostingsBenchmark.buildPostings`	100 books	1853.0 ms/op
`JsonIndexBenchmark.serializeIndex`	100-book index	500.6 ms/op
`JsonIndexBenchmark.parseIndex`	100-book index	114.3 ms/op
`QueryBenchmark.intersectQuery`	two-term intersection	2.8 μs/op

How these numbers may be used. They explain the *internal* split of one Java operation (for example, serialising the JSON index dominates parsing it by roughly 4.4×), which helps attribute the end-to-end Java bars of section 8. They are **evidence about the Java port only**: JMH's forked-JVM warm-up protocol is deliberately *not* applied to the other languages, so no JMH number is ever placed next to a Python, Go or Rust measurement in this report. The cross-language figures of section 6 all come from the common harness of section 4.

10 Conclusions and Final Architecture Selection

What the measurements support. On the host of table 1, over the 100-book corpus and the 30-query workload, the tournament measured the following:

- The four implementations are behaviourally identical on this workload (byte-identical canonical outputs, section 5), so the cross-language differences below compare the same work.
- On write throughput and datalake lookup, the measured spread between implementations is 2.6–2.8× in write throughput (27.3–82.8 items/s across layouts) and 6–13× in lookup latency, though every lookup stays below 0.3 ms.

- On index build time and query latency, Go built the 10,000-book JSON index in 66.0 s against Java's 947.0 s, while query latency stayed between 0.6 and 5.0 ms across structures and languages; on incremental updates over the measured base sizes, the JSON update grew 16–23× with the base size while the folder and MongoDB updates grew only 1.4–2.8×.
- On-disk footprint is dominated by the corpus itself; the storage overhead of the three index structures over the same corpus is 75.9 MB as a single JSON file, 42.1 MB across 106,479 files for the folder structure, and 38.2–558.9 MB server-side for MongoDB (`collStats`) (section 8.4).
- Recovery after a documented interruption resumed with 0 duplicated and 0 lost books in every layout (table 8).

Selection for Stage 2. Within the tested corpus, workload and host, the group selects **Java with the MongoDB index structure** for Stage 2. The reasoning is scoped to the evidence: MongoDB showed near-flat update scaling (1.4–2.8× growth while the base index grew 40×) and a footprint of several hundred megabytes at the measured sizes, its server-side footprint was measured separately (`collStats`), and the Java port's end-to-end numbers were supported by JMH attribution. This is a choice *for the Stage 2 workload*, not a claim that Java is universally faster or that MongoDB is the only scalable storage: at the measured scale, the file-based structures were competitive on several axes, and behaviours that matter at larger scale — sharding, replication, concurrency, multi-gigabyte indexes — were *not measured here* and remain open questions (section 11).

Language-wide claims. Nothing in this report supports a statement of the form "language X is faster than language Y". The measured objects are four implementations, written by this team, of one pipeline, on one corpus, on one machine, in one continuous run. Statements such as "Go's allocator showed the lowest RSS growth in this workload" are observations; "Go uses less memory" is a hypothesis this experiment cannot test. Where the data was ambiguous, the report says so rather than picking a narrative.

10.1 Future Improvements

- Repeat the tournament on bare metal and on Linux natively to separate runtime effects from WSL2's virtualisation.
- Extend the corpus by an order of magnitude to observe where the measured curves bend (the current 10,000-book batch reuses a 100-book corpus, section 4.3).
- Measure concurrency (multiple query clients) for the Stage 2 serving layer, where it will actually be used.
- Add a memory-profiled MongoDB run that accounts the server's RSS alongside the client's, for a true capacity figure.

11 Limitations

Every observed difference could partly be an artefact of the following factors. They are listed so readers can weigh the results accordingly.

Machine. One laptop-class host (i7-10750H, 8 GB RAM), under WSL2 virtualisation, with an NTFS-backed filesystem. CPU frequency scaling, thermal throttling and the hypervisor layer affect all languages, but not necessarily equally. No isolation (no `taskset`, no performance governor, no bare metal) was used.

Runtime. The Java harness runs on OpenJDK 17 through `mvn exec:java` (Maven's own JVM overhead is outside the timed regions but shares the machine), while JMH runs on 21. Python runs on 3.14, Go 1.27, Rust 1.99: these are current versions, not long-term-stability comparisons. Only Java receives warm-up (section 4.4); the other languages are measured from cold start, which is the honest default for them but is itself a protocol choice.

Cache state. The OS page cache is never dropped between phases, so a file read twice in one phase may be served from RAM. All languages in a phase share the same state, but phases are not independent of the ones before them. MongoDB's internal cache (WiredTiger) is likewise warm after the first build.

Workload. The corpus is 100 books (61.4 MB); the scalability builds reach 10,000 "books" by repeating that corpus, so index size grows but term diversity does not. Queries are 30 fixed lines; they are representative of the assignment but not sampled from a query log. Results may reorder on corpora with different term distributions.

Timing and statistics. Elapsed times are wall-clock in a shared, unisolated host: neighbouring processes (including `mongod` and the sampling threads) inject jitter, visible as non-zero `stdev_seconds`. One tournament run produces the committed snapshot; there is no second run to quantify run-to-run variance. No outliers are trimmed and no phase is repeated for a better result.

Measurement boundaries. Client RSS excludes the MongoDB server by construction (section 8.4); its storage is measured via `collStats`, its memory is not. The download probe is ten sequential requests, so its rate reflects latency more than bandwidth. CPU percentage is process CPU over wall clock and can exceed 100 on multi-core bursts.

Unmeasured behaviours. Not exercised at all: concurrent readers and writers, indexes far beyond the measured batches, MongoDB replication/sharding, crash recovery of the database itself, and the Stage 2 serving path. Conclusions in section 10 do not extend to them.

12 Reproducibility

Every figure in this report is reproducible from the repository at a single identified revision. The full protocol is documented in `REPRODUCING.md`; the short version follows.

One command.

```
./scripts/run_tournament.sh
```

runs five phases in order and refuses to time anything before equivalence passes:

1. **equivalence** — cross-language canonical files, compared byte for byte (`equivalence_report.json`);
2. **benchmarks** — Go, Rust, Java (+JMH) and Python suites, each writing its CSVs under `benchmarks_results/<language>/`;
3. **verify** — schema check of every CSV (`scripts/verify_format.py`);
4. **environment** — host, runtimes and Git revision in `benchmarks_results/environment.txt`;
5. **charts** — `src/visualizations/main.py` renders `charts/` from the CSVs.

There is no resume: a failed run restarts clean, so the committed snapshot always comes from one continuous execution.

Chart-to-artifact map. Table 10 links every figure to its script (all chart scripts live under `src/visualizations/modules/`), its raw results and its configuration.

Table 10: Where each figure comes from.

Figure	Script	Raw results	
fig. 2	`radar_chart.py`	all of `benchmarks_results/`	
fig. 3	`throughput_charts.py`	`/throughput.csv`, `/benchmarks.csv`	×
fig. 4	`latency_charts.py`	`/statistics.csv`	
figs. 5 to 7	`scalability_charts.py`	`/scalability.csv`, `/updates.csv`	×
Sankey (interactive HTML)	`storage_sankey.py`	`/disk_usage.csv`	
table 9	`multilang_ports/java/run_jmh.sh`	`java/jmh_results.json`	

Revision. The Git commit that produced the committed snapshot is recorded inside `benchmarks_results/en` together with the host and runtime versions of table 1. Checking out that commit and re-running the command above reproduces the pipeline; the raw numbers will differ in their last digits (shared-host jitter, section 11), but the ordering and the equivalence digests are stable.

13 Correction Comment Matrix

Table 11 maps each of the 33 correction comments to the place in this report (or in the repository) that answers it.

Table 11: Correction comments mapped to the place in this report (or in the repository) that answers each one.

#	Topic	Where it is addressed
1	Scope language/storage claims to the workload	sections 1 and 10: claims scoped to corpus, host and workload.
2	Host, runtimes, execution parameters, download concurrency	tables 1 and 3 and section 4.5: downloads sequential, no concurrency anywhere.
3	RSS-delta definition, 1,601 MB, O(1) MongoDB	section 8.1: delta definition; the old 1,601 MB figure re-stated as the largest plotted delta (1,028.5 MB in this snapshot); section 8.3 replaces the O(1) claim.
4	Pipeline diagram of the three layers	fig. 1: TikZ diagram, layers, data movement, durable-write points.
5	Radar productivity axis, “Peak” callout	section 6.1 and fig. 6: qualitative score labelled in the figure; callout renamed “Largest RSS delta”.
6	Discovery, storage overhead, recovery observations	sections 7.3 to 7.5, from `disk_usage.csv` and `recovery.csv`.
7	Normalisation/metadata rules, postings examples, all languages	sections 3.1 and 3.2: ordered rules and serialised postings example; per-language check in section 5.1.
8	Link charts to script, config, raw results, revision	table 10 and section 4.9 plus `environment.txt`.
9	Limitations section	section 11.
10	Do not call MongoDB “the only scalable choice”	section 10: selection justified for the Stage 2 workload only.
11	Delta vs peak RSS, RAM-spike/GC evidence, O(1) variable	section 8.1: `peak_rss_mb` sampling and Go `memstats` as evidence; section 8.3 defines operation and variable.
12	Compare metadata, terms, postings, answers; equivalence first	section 5: gates run before any timing; outputs byte-identical.

#	Topic	Where it is addressed
13	Record host/runtimes/concurrency per language, align or restrict	tables 1, 3 and 4 and section 4.4: identical parameters; Java warm-up stated.
14	Corpus, operation and timing boundaries behind plots	sections 4.3, 4.6 and 4.8.
15	Reinterpret RAM/timing; language explanations as hypotheses	sections 6.2 and 10: explicit reading rule.
16	Book counts, sizes, timing start/stop for write and download	sections 4.6 and 7.1.
17	Discovery of downloaded-but-not-indexed books, same state	section 7.3: `detection_time` per layout, identical initial state.
18	Files, directories, auxiliary state, bytes per layout	section 7.4 and table 7 from `disk_usage.csv`.
19	Directory counts before choosing batches; complement lookup	sections 7.4 and 8.2: counts reported and tied to the lookup differences.
20	Isolated incremental-update experiment	section 8.3 and fig. 7: untimed base of 25–1000 books, +5 books, 5 repetitions, `updates.csv`.
21	Publish workload and expected answers; cross-structure check	`src/utils/benchmarks/queries.txt` with semantics in table 3; verification in section 5.2 (`test_structure_equivalence.py`).
22	Sampling procedure, processes included, peak RSS, MongoDB server	table 2 and section 8.4: 50 ms sampling of the client; server via `collStats`.
23	Support MongoDB with measured updates/space; drop O(1)	sections 8.3 and 8.4 and fig. 7: O(1) replaced by the observed curve.
24	Demonstrated interruption and resumption with counts	section 7.5 and table 8: 50/100 books, duplicated and lost IDs reported.
25	Compare IDs, metadata, postings, answers; also after updates	section 5.2: repeated after an update.
26	Limit Java/MongoDB conclusions; state unmeasured behaviours	sections 10 and 11: selection scoped to the tested corpus.
27	CPU, RAM, storage, OS, exact runtime versions	table 1 plus `benchmarks_results/environment.txt`.
28	Corpus IDs/sizes, queries, update batch, coverage	tables 3 and 4 plus `equivalence_report.json` (IDs); update bases 25–1000 +5.
29	Output preparation, cache state, network, warm-up	sections 4.4 and 4.5: fresh tree per phase, Java-only warm-up.
30	Sample count, clock, timing boundaries, repetition procedure	tables 2 and 3 and sections 4.6 and 4.7: 150 query samples, monotonic clocks, stdev definition.
31	Exact commands and raw-result locations per chart, revision	section 12 and table 10 plus `environment.txt`.
32	Pipeline quality; Stage 2 search not required	No change needed: user-facing search belongs to Stage 2; term lookup exists (section 2).
33	Code quality; no deduction for Stage 2 or optional packages	No change needed: optional dependencies in `REPRODUCING.md`.